Solution Architecture

Date	30 June2026
Team ID	
Project Name	Rising Waters: Smart AI Flood Prediction System
Maximum Marks	5 Marks

Solution Architecture Diagram:
Provide a high-level visual representation of your solution's architecture. Use the structural layout below as a template to map out your Presentation Layer, Application Layer, and Data Layer. Fill in the specific technologies and components for your project directly into the boxes to create your architectural diagram.

Presentation / Client Layer
[e.g., Web App (React), Mobile App (Flutter), Admin Dashboard]

↓↑

API Gateway / Load Balancer
[e.g., Nginx, AWS API Gateway, Express Server]

↓↑

Auth Service	**Core Logic Service**	**External API Integrations**
[e.g., JWT, OAuth2]	*[e.g., Data Processing, Machine Learning Models]*	*[e.g., Payment Gateway, Email Service]*

↓↑

Data / Storage Layer
[e.g., PostgreSQL Database, MongoDB, AWS S3]

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides the user interface where users enter weather parameters (such as rainfall and cloud visibility) and view flood prediction results. Ensures an interactive and responsive experience.	HTML5, CSS3, JavaScript, Bootstrap 5
Application Layer (Flask Server)	Receives user inputs from the frontend, processes prediction requests, loads the trained machine learning model, and returns flood prediction results to the user.	Python 3.11, Flask
Microservices	Performs data preprocessing, feature scaling, and flood prediction using trained classification algorithms. The XGBoost model is selected as the final deployed model due to its high accuracy.	Scikit-learn, XGBoost, NumPy, Pandas, Joblib
Database	Stores the historical weather dataset used for model training and the serialized trained model used during prediction. Optionally stores prediction history if required.	CSV Dataset, Joblib (.save/.pkl), SQLite (Optional)